

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 16 | Graphs: Representation, BFS, DFS, Topological Sort and Shortest Paths

Week 16 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 16 at a glance

Item	Details
Lab title	Graphs: Representation, BFS, DFS, Topological Sort and Shortest Paths
Week	16 of 16
Prerequisites	Lab 10 (Stacks), Lab 11 (Queues), Lab 12 (Hashing), Lab 15 (Heaps)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab16_Instructor.ipynb , Lab16_Student.ipynb

Item	Details
Next lab	Course complete! See “Where to Go Next” in this manual.

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain graph terminology: **vertex**, **edge**, **directed**, **undirected**, **weighted**, **degree**, **path**, **cycle**, **connected**, **DAG**.
2. Represent graphs with an **adjacency matrix**, an **adjacency list** and an **edge list**, and compare them.
3. Implement **Breadth-First Search (BFS)** and find shortest paths in unweighted graphs.
4. Implement **Depth-First Search (DFS)** recursively and iteratively.
5. Find **connected components** and detect **cycles**.
6. Order tasks with **topological sort** (Kahn's algorithm).
7. Find shortest paths in weighted graphs with **Dijkstra's algorithm** using a heap.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain graph representations and terminology	Understand (C2)
CLO-2	Implement graph traversal and shortest-path algorithms	Apply (C3)
CLO-3	Model real problems as graphs and choose suitable algorithms	Analyse / Create (C4–C6)

2. Motivation: Everything Is Connected

Trees model hierarchies, but many relationships are not hierarchical. Cities are connected by roads in all directions; people follow each other on social media; web pages link to each other. A **graph** is the most general data structure for **connections**, and it brings together everything you learned this semester: lists, stacks, queues, hash tables and heaps.

A road network is a weighted graph (distances are illustrative)

```

Okara ---- 110 km ---- Lahore
 |      \                /      |
60 km   \ 125 km       45 km  50 km
 |      \                /      |
Sahiwal  Faisalabad ---- Sheikhpura
  
```

2.1 Graphs in Daily Life

Graph	Vertices	Edges
Google Maps / Careem	Places, intersections	Roads (weighted by distance or time)
Facebook / LinkedIn	People	Friendships / connections (undirected)
Instagram / X (Twitter)	Accounts	“Follows” (directed)
Your degree plan	Courses	Prerequisites (directed, no cycles)
Metro / bus routes (Lahore Orange Line)	Stations	Track segments
The Internet	Routers and computers	Cables and wireless links

2.2 Graphs in Industry

- **Navigation:** Google Maps, Waze and ride-hailing apps compute shortest and fastest routes with Dijkstra and A*.
- **Social networks:** friend suggestions (“people you may know”) explore friends-of-friends with BFS.
- **Search engines:** Google’s original **PageRank** ranks web pages using the link graph.
- **Build systems and package managers** (pip, npm, Make) order tasks with topological sort.
- **Telecom and electricity networks** plan cables with minimum spanning trees; **fraud detection** finds suspicious rings of accounts as cycles.

2.3 Graphs in Research

- **Graph Neural Networks (GNNs)** learn from molecules, social networks and road traffic.
- **Knowledge graphs** give AI assistants structured facts; vector search uses graph indexes (HNSW).
- **Deep learning frameworks** (PyTorch, TensorFlow) represent every model as a **computation graph** and run backpropagation in topological order.
- **Remote Sensing & GIS:** road extraction from satellite images, river networks and flood routing are graph problems.

- **Bioinformatics:** protein-interaction networks and genome assembly (de Bruijn graphs).

2.4 Why It Matters in 2026–2030

Graphs are the language of connected data

Recommendation systems, logistics, supply chains, AI agents planning steps, and neural networks themselves are all graphs. Graph algorithms are among the most valuable skills for AI, data science and systems engineering in the coming years, and graph problems are a favourite of technical interviews.

3. Part A: Graph Terminology

A **graph** $G = (V, E)$ is a set of **vertices** (nodes) V and a set of **edges** E connecting pairs of vertices.

Term	Meaning
Undirected graph	Edges have no direction ($A - B$ means both ways), e.g., friendship
Directed graph (digraph)	Edges have a direction ($A \rightarrow B$), e.g., following, prerequisites
Weighted graph	Each edge has a cost/weight, e.g., distance, time, price
Adjacent / neighbours	Two vertices joined by an edge
Degree	Number of edges at a vertex (in-degree / out-degree for directed graphs)
Path	A sequence of vertices connected by edges
Cycle	A path that starts and ends at the same vertex
Connected graph	There is a path between every pair of vertices (undirected)
Connected component	A maximal group of vertices that are connected to each other
DAG	Directed Acyclic Graph: directed, with no cycles (e.g., course prerequisites)
Tree	A connected undirected graph with no cycles (n vertices, $n - 1$ edges)



4. Part B: Representing a Graph

Example (undirected):

```

    0 --- 1
    |   / |
    |   / |
    2 --- 3      4 (isolated)
  
```

Edges: (0,1) (0,2) (1,2) (1,3) (2,3)

4.1 Adjacency Matrix

An $n \times n$ table where $M[u][v] = 1$ (or the weight) if there is an edge $u-v$. Checking an edge is $O(1)$, but it uses $O(V^2)$ memory even if there are few edges.

4.2 Adjacency List

For each vertex, store a **list of its neighbours** (in Python, a dictionary of lists). Memory is $O(V + E)$; this is the standard choice for real (sparse) graphs.

```

n = 5
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3)]

matrix = [[0] * n for _ in range(n)]
adj = {v: [] for v in range(n)}
for u, v in edges:
    matrix[u][v] = matrix[v][u] = 1          # undirected: both directions
    adj[u].append(v)
    adj[v].append(u)

print("Adjacency matrix:")
for row in matrix:
    print(" ", row)
print("Adjacency list:", adj)
print("degree of 1:", len(adj[1]), "| edge 0-3?", matrix[0][3] == 1)
  
```

	Adjacency matrix	Adjacency list
Memory	$O(V^2)$	$O(V + E)$
Check if edge $u-v$ exists	$O(1)$	$O(\text{degree of } u)$
List all neighbours of u	$O(V)$	$O(\text{degree of } u)$
Best for	Dense graphs, small V	Sparse graphs (roads, social networks)

5. Part C: Breadth-First Search (BFS)

BFS explores the graph **level by level**: first the start vertex, then all its neighbours, then their neighbours, and so on. It uses a **queue** and a **visited** set. In an unweighted graph, BFS finds the **shortest path (fewest edges)** from the start to every vertex.

Pseudocode

```
BFS(G, s):  
    visited ← {s} ; queue ← [s] ; dist[s] ← 0  
    WHILE queue not empty:  
        u ← dequeue()  
        FOR each neighbour v of u:  
            IF v not in visited:  
                visited.add(v) ; dist[v] ← dist[u] + 1 ; parent[v] ← u  
                enqueue(v)
```

```

from collections import deque

city = {
    "Okara": ["Sahiwal", "Lahore", "Faisalabad"],
    "Sahiwal": ["Okara", "Multan"],
    "Lahore": ["Okara", "Sheikhupura", "Gujranwala"],
    "Faisalabad": ["Okara", "Sheikhupura"],
    "Sheikhupura": ["Lahore", "Faisalabad"],
    "Gujranwala": ["Lahore"],
    "Multan": ["Sahiwal"],
}

def bfs(graph, start):
    dist, parent = {start: 0}, {start: None}
    q = deque([start])
    while q:
        u = q.popleft()
        for v in graph[u]:
            if v not in dist:                # dist doubles as the visited
                dist[v] = dist[u] + 1
                parent[v] = u
                q.append(v)
    return dist, parent

def path_to(parent, target):
    path = []
    while target is not None:
        path.append(target)
        target = parent[target]
    return path[::-1]

dist, parent = bfs(city, "Multan")
print(dist)
print("Route Multan → Gujranwala:", " → ".join(path_to(parent,
"Gujranwala")))

```

6. Part D: Depth-First Search (DFS)

DFS goes **as deep as possible** along one branch before backtracking, like exploring a maze by always taking the first unexplored corridor. It uses **recursion** (the call stack) or an explicit **stack**.

```

def dfs_recursive(graph, u, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(u)
    order.append(u)
    for v in graph[u]:
        if v not in visited:
            dfs_recursive(graph, v, visited, order)
    return order

def dfs_iterative(graph, start):
    visited, order, stack = set(), [], [start]
    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        for v in reversed(graph[u]):           # reversed: same order as
recursive
            if v not in visited:
                stack.append(v)
    return order

print("DFS (recursive):", dfs_recursive(city, "Okara"))
print("DFS (stack)      :", dfs_iterative(city, "Okara"))

```

	BFS	DFS
Data structure	Queue	Stack / recursion
Order	Level by level	Deep first, then backtrack
Shortest path (unweighted)	Yes	No
Typical uses	Shortest routes, friend suggestions, broadcasting	Cycle detection, topological sort, mazes, components
Time	$O(V + E)$	$O(V + E)$

6.1 Connected Components and Cycle Detection

```
def components(graph):
    seen, groups = set(), []
    for v in graph:
        if v not in seen:
            group = dfs_recursive(graph, v, seen, [])
            groups.append(sorted(group))
    return groups

def has_cycle_undirected(graph):
    seen = set()
    def dfs(u, parent):
        seen.add(u)
        for v in graph[u]:
            if v not in seen:
                if dfs(v, u):
                    return True
            elif v != parent:
                # visited and not the edge we came from
                return True
    return False
    return any(dfs(v, None) for v in graph if v not in seen)

friends = {"Ali": ["Sara"], "Sara": ["Ali", "Hamza"], "Hamza": ["Sara"],
          "Zoya": ["Iqra"], "Iqra": ["Zoya"], "Omar": []}
print("friend groups:", components(friends))
print("cycle in city graph?", has_cycle_undirected(city), "| in friends?",
      has_cycle_undirected(friends))
```



7. Part E: Topological Sort (Ordering Tasks)

In a **DAG**, a topological order lists the vertices so that every edge $u \rightarrow v$ has u **before** v . Example: you must pass Programming Fundamentals before OOP, and OOP before DSA. **Kahn's algorithm**: repeatedly take a vertex with **in-degree 0** (no remaining prerequisites), output it, and remove its outgoing edges. If some vertices are never output, the graph has a **cycle** (impossible plan).

```
def topological_sort(graph):
    indegree = {v: 0 for v in graph}
    for u in graph:
        for v in graph[u]:
            indegree[v] += 1
    q = deque(sorted(v for v in graph if indegree[v] == 0))
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                q.append(v)
    return order if len(order) == len(graph) else None    # None = cycle

courses = {
    "PF": ["OOP", "DLD"], "OOP": ["DSA"], "DLD": ["COAL"], "Discrete":
["DSA"],
    "DSA": ["Algorithms", "DB"], "COAL": ["OS"], "Algorithms": [], "DB": [],
    "OS": [],
}
print("Study order:", " → ".join(topological_sort(courses)))
```



8. Part F: Dijkstra's Shortest Path (Weighted Graphs)

When edges have weights (distances, times, costs), BFS no longer gives the shortest path. **Dijkstra's algorithm** always expands the **closest unexplored vertex**, which it gets from a **min-heap**. It works when all weights are **non-negative**. Time: $O((V + E) \log V)$.

Pseudocode

```
Dijkstra(G, s):
    dist[v] ← ∞ for all v ; dist[s] ← 0 ; heap ← [(0, s)]
    WHILE heap not empty:
        (d, u) ← pop smallest
        IF d > dist[u]: CONTINUE                // outdated entry
        FOR each (v, w) in neighbours of u:
            IF dist[u] + w < dist[v]:           // relaxation
                dist[v] ← dist[u] + w ; parent[v] ← u ; push (dist[v], v)
```

```

import heapq

roads = {
    # km (illustrative)
    "Okara": [("Lahore", 110), ("Sahiwal", 60), ("Faisalabad", 125)],
    "Lahore": [("Okara", 110), ("Sheikhupura", 45)],
    "Sahiwal": [("Okara", 60), ("Faisalabad", 115)],
    "Faisalabad": [("Okara", 125), ("Sahiwal", 115), ("Sheikhupura", 90)],
    "Sheikhupura": [("Lahore", 45), ("Faisalabad", 90)],
}

def dijkstra(graph, source):
    dist = {v: float("inf") for v in graph}
    parent = {source: None}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (dist[v], v))
    return dist, parent

dist, parent = dijkstra(roads, "Sahiwal")
for city_name, d in sorted(dist.items(), key=lambda kv: kv[1]):
    print(f"{city_name:<12} {d:>4} km    via {' → '.join(path_to(parent, city_name))}")

```

Problem	Algorithm	Time
Visit all vertices / reachability	BFS or DFS	$O(V + E)$
Shortest path, unweighted	BFS	$O(V + E)$
Shortest path, non-negative weights	Dijkstra (heap)	$O((V + E) \log V)$
Shortest path with negative weights	Bellman-Ford	$O(V \cdot E)$
Ordering with dependencies	Topological sort	$O(V + E)$
Connect all vertices at minimum cost	Prim / Kruskal (MST)	$O(E \log V)$

9. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <queue>
#include <limits>
using namespace std;

int main() {
    int n = 5;
    vector<vector<pair<int,int>>> adj(n);          // adjacency list: (neighbour, weight)
    auto addEdge = [&](int u, int v, int w) { adj[u].push_back({v, w}); adj[v].push_back({u,
w}); };
    addEdge(0, 1, 4); addEdge(0, 2, 1); addEdge(2, 1, 2); addEdge(1, 3, 5); addEdge(3, 4,
3);

    // BFS (ignoring weights)
    vector<int> level(n, -1); queue<int> q; q.push(0); level[0] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto [v, w] : adj[u]) if (level[v] == -1) { level[v] = level[u] + 1; q.push(v);
    }
    }

    // Dijkstra
    vector<int> dist(n, INT_MAX); dist[0] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; pq.push({0, 0});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    for (int i = 0; i < n; i++) cout << i << ": level " << level[i] << ", dist " << dist[i]
<< endl;
}
```

10. Common Mistakes

Mistake	Effect	Fix
Forgetting the visited set	Infinite loop on cycles	Mark vertices visited when they are discovered
Marking visited when dequeued (BFS)	Vertices enqueued many times	Mark when enqueued
Adding only one direction for undirected edges	Missing paths	Add $u \rightarrow v$ and $v \rightarrow u$
Using BFS for weighted shortest paths	Wrong distances	Use Dijkstra
Dijkstra with negative weights	Wrong answers	Use Bellman-Ford

Mistake	Effect	Fix
Deep recursion in DFS on big graphs	RecursionError	Use the iterative stack version

11. Course Wrap-Up: What You Have Built

Congratulations! In 16 weeks you went from writing your first Python function to implementing the data structures that power operating systems, databases, search engines and AI systems.

Weeks	You learned	Real-world power
1–2	Python, Big-O	Judge the efficiency of any code, human or AI-written
3–7	Arrays, searching, sorting, recursion	Fast search and ordering of data
8–11	Linked lists, stacks, queues	Undo, browsers, schedulers, caches
12	Hashing	O(1) lookup behind dicts, caches and databases
13–15	Trees, BSTs, heaps	Indexes, ordered maps, priority systems
16	Graphs	Maps, social networks, dependencies, AI planning

11.1 Capstone Project (Choose One, Teams of 2–3)

1. **Smart Campus Navigator:** model University of Okara buildings as a weighted graph; shortest walking routes (Dijkstra), accessible routes (avoid stairs), and a list of nearest facilities (heap).
2. **Mini Search Engine:** index a folder of text documents with a hash map (word → documents), rank results by frequency with a heap, and support prefix auto-complete (sorted list + binary search, or a trie).
3. **Course Planner:** read the department's courses and prerequisites, detect impossible plans (cycles), produce a valid semester-by-semester plan (topological sort by levels).
4. **Hospital / Bank Simulator:** queues and priority queues with arrival times, multiple servers and statistics, plus an LRU cache for patient records.
5. **Your own idea** that uses at least **four** different data structures from this course (approved by your instructor).

Deliverables: GitHub repository with code and README, a 5-minute demo, and a short report explaining which data structures you chose and the **Big-O** of the main operations.



11.2 Where to Go Next

- **Algorithms course:** dynamic programming, greedy algorithms, minimum spanning trees, network flow, NP-completeness.
- **Advanced data structures:** tries, segment trees, Fenwick trees, union-find (disjoint sets), B-trees, skip lists, Bloom filters.
- **Practice:** aim for 150+ LeetCode problems (e.g., the “NeetCode 150” or “Blind 75” lists) before internship interviews.
- **Apply it:** NumPy/Pandas (arrays), databases (B-trees, hashing), PyTorch (computation graphs), and graph libraries like NetworkX.



12. Lab Tasks

Solve all tasks in `Lab16_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Degrees and the Class Monitor

In a class of n students ($0 \dots n-1$), `trust = [(a, b), ...]` means student a trusts student b (directed). The **monitor** is trusted by everyone else and trusts nobody. Return the monitor's number or -1 .

Hint: compute in-degree and out-degree for every student. The notebook provides `build_graph(n, edges, directed)`.

```
n = 3, trust = [(0, 2), (1, 2)] → 2
```

Task 2 Is There a Route?

Given an undirected graph with n vertices and an edge list, return `True` if there is a path from `source` to `target`. Solve it with DFS using an explicit stack.

```
n = 6, edges = [(0,1),(0,2),(3,5),(5,4),(4,3)], 0 → 5 → False
```

Task 3 Shortest Route with BFS

Return the **actual shortest path** (list of vertices) from `start` to `goal` in an unweighted undirected graph, or `[]` if unreachable. Use BFS with a `parent` dictionary.

```
edges [(0,1),(1,2),(0,3),(3,4),(4,2)], 0 → 2 → [0, 1, 2]
```

Task 4 Count Flooded Regions (Connected Components on a Grid)

A flood map from satellite imagery is a grid of "W" (water) and "." (dry). Water cells connected up/down/left/right form one flooded region. Count the regions and return the **size of the largest** one as `(count, largest)`.

Use BFS from every unvisited water cell (each cell is a vertex; neighbours are adjacent cells).

```
["WW..", "W..W", "..WW", "...."] → (2, 3)
```

Task 5 🟡 Can I Finish My Degree? (Course Schedule)

There are n courses ($0 \dots n-1$) and prerequisite pairs (a, b) meaning **b must be taken before a**. Return a valid order of all courses, or $[]$ if it is impossible because of a cycle.

Use Kahn's topological sort. To make the answer deterministic, always take the **smallest** available course first (use a heap instead of a queue).

```
n = 4, prereqs = [(1, 0), (2, 0), (3, 1), (3, 2)] → [0, 1, 2, 3]
```

Task 6 🔴 Network Delay Time (Dijkstra)

A message is sent from server k in a network of n servers ($1 \dots n$). $times = [(u, v, w), \dots]$ means a directed link from u to v taking w milliseconds. Return the time for **all** servers to receive the message, or -1 if some cannot be reached.

Run Dijkstra from k ; the answer is the maximum shortest distance.

```
times = [(2,1,1), (2,3,1), (3,4,1)], n = 4, k = 2 → 2
```

Task 7 🔴 Spread of Crop Disease (Multi-Source BFS)

A field is a grid: 0 empty, 1 healthy plant, 2 infected plant. Every day, each infected plant infects its healthy neighbours (up/down/left/right). Return the number of days until no healthy plant remains, or -1 if some plants can never be infected.

Start BFS from **all** infected plants at once (put them all in the queue with day 0).

```
[[2,1,1],[1,1,0],[0,1,1]] → 4
```



13. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab16** folder.

#	Problem	Level	Concept practised
997	Find the Town Judge	Easy	Degrees (Task 1)
1971	Find if Path Exists in Graph	Easy	DFS / BFS (Task 2)
733	Flood Fill	Easy	Grid traversal
200	Number of Islands	Medium	Components (Task 4)
207	Course Schedule	Medium	Topological sort (Task 5)

#	Problem	Level	Concept practised
133	Clone Graph	Medium	BFS/DFS with a hash map
994	Rotting Oranges	Medium	Multi-source BFS (Task 7)
743	Network Delay Time	Medium	Dijkstra (Task 6)



14. Home Task

1. Finish all tasks and push `Lab16_Student.ipynb` to GitHub.
2. Implement cycle detection in a **directed** graph using DFS with three colours (white, grey, black).
3. Implement **Prim's** or **Kruskal's** algorithm to connect a set of villages with roads at minimum total cost.
4. Start your **Capstone Project** (see the Course Wrap-Up section): form a team, choose a topic, and create the GitHub repository with a README describing the data structures you plan to use.
5. Solve at least 3 LeetCode problems from the list.

? 15. Viva Questions

1. Define a graph. What is the difference between directed and undirected graphs?
2. What is a weighted graph? Give a real example.
3. Compare adjacency matrix and adjacency list representations.
4. Explain BFS. Which data structure does it use and why does it find shortest paths in unweighted graphs?
5. Explain DFS. How is it different from BFS?
6. What is the time complexity of BFS and DFS with an adjacency list?
7. What is a connected component? How do you count them?
8. What is a DAG? What is a topological order?
9. Explain Dijkstra's algorithm. Why does it need a priority queue?
10. Why does Dijkstra fail with negative edge weights?



16. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



17. References and Further Learning

- Graph algorithm animations: visualgo.net/en/graphds, visualgo.net/en/dfsdfs, visualgo.net/en/sssp
- NetworkX documentation: networkx.org
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.